WEEK 9

Week 9: Accelerators - Lab Guide

HPC & Cluster Computing

Objective: Verify GPU availability and compile a basic CUDA application. **Goal**: Understand the Host-Device workflow.

---

1. Prerequisites

- [ ] A node with an NVIDIA GPU (or a simulated environment).
- [ ] `cuda-toolkit` installed (e.g., `/usr/local/cuda`).
- [ ] `gcc` compiler.

---

2. Infrastructure Check

2.1 Check Hardware

The command `nvidia-smi` (System Management Interface) is the standard tool to query GPU status.

- [ ] Run the query: `bash nvidia-smi`

- **Memory Usage**: Is the VRAM empty?

2.2 Monitor in Real-time

To watch the GPU load (like `top` for CPUs):

- [ ] Start the monitor:

```
bash watch -n 1 nvidia-smi
```

---

3. The "Vector Add" Application

We will write a simple program that adds two arrays of 100,000 numbers. One version runs on the CPU, the other on the GPU.

3.1 Create `vec_add.cu`

- [ ] Create a file named `vec_add.cu` using `nano`: `bash nano vec_add.cu`
- [ ] Paste the following content: ```cpp #include #include

  // The "Kernel" - Runs on the GPU **global** void vectorAdd(int *a, int* b, int *c, int N) { int i = blockIdx.x * blockDim.x + threadIdx.x; if (i < N) { c[i] = a[i] + b[i]; } }

  int main() { int N = 100000; int size = N * sizeof(int);

```
// Allocate Host Memory
int *h_a, *h_b, *h_c;
cudaMallocHost(&h_a, size);
cudaMallocHost(&h_b, size);
cudaMallocHost(&h_c, size);
```

```
    for(int i=0; i<N; i++) { h_a[i] = i; h_b[i] = i; }

    // Allocate Device Memory
    int *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, size);
    cudaMalloc(&d_b, size);
    cudaMalloc(&d_c, size);

    // Copy to Device
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    // Launch Kernel (N/256 blocks, 256 threads per block)
    vectorAdd<<<(N+255)/256, 256>>>(d_a, d_b, d_c, N);

    // Copy back
    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    // Verify
    printf("Success! Last element: %d + %d = %d\n", h_a[N-

    // Cleanup
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    cudaFreeHost(h_a); cudaFreeHost(h_b); cudaFreeHost(h_c
    return 0;
```

}```

4. Compilation and Execut

- [ ] Compile the code:

```
bash /usr/local/cuda/bin/nvcc -o vec_add vec_add.cu
```

4.2 Run

- [ ] Execute the binary: `bash ./vec_add`
- [ ] **Verification**
  - Expected Output: `Success! Last element: 99999 + 99999 = 199998`

4.3 Reflection

- [ ] **Concept Check**: Why did we need `cudaMemcpyHostToDevice`? Could the GPU read the `h_a` variable directly? (Hint: They have separate RAM).

---

5. Submission

Submit a screenshot of:

- [ ] The output of `nvidia-smi`.
- [ ] The output of your `./vec_add` run.